CESAR SCHOOL · INFRAESTRUTURA DE SOFTWARE · 2026.2

Rota Vital

Container, TDD e deploy automatizados no Projeto Integrador

Cauã Rego Tavares Leite Duarte · Fernando Andrade Leandro Peixoto · Gabriel Brito Ferreira Dias · Guilherme Alves de Souza · Lucas Ferreira Torres de Albuquerque · Maria Eduarda Vasconcelos · Renato Santos Chong · Victor Barros Roma

A PERGUNTA DO DESAFIO

“Se um colega clonar o repositório amanhã, ele consegue rodar, testar e publicar o sistema sem pedir ajuda?”

E1

Container

O sistema sobe do zero com um comando, igual em qualquer máquina.

E2

TDD

As regras de negócio protegidas por testes em três níveis, rodando na esteira.

E3

Deploy

Cada merge na main chega sozinho ao ar, e só com a esteira verde.

E1 · CONTAINER AUTOMATIZADO

Um comando sobe API e banco do zero

```
$ docker compose up -d --build
```



- **Multi-stage:** o 1º estágio compila e roda os testes; o 2º leva só o jar
- **Healthcheck** nos dois serviços; a API espera o banco ficar pronto
- **Volume nomeado:** os dados sobrevivem ao compose down
- **Rede própria:** a API acha o banco pelo nome
- **.env** fora do git; nenhum segredo no repositório

E2 · TDD · STORY MAP

A jornada da bolsa, da coleta à rede

	1. Abastecer o estoque	2. Pedir sangue	3. Separar as bolsas	4. Entregar	5. Acompanhar a rede
Release 1	HU-01 Cadastrar bolsa coletada	HU-02 Solicitar hemocomponentes	HU-03 Alocar por validade (FEFO) · HU-08 Alocar respeitando ABO/Rh	HU-05 Calcular rota de entrega · HU-07 Monitorar temperatura no transporte	HU-04 Bolsas em risco · HU-06 Indicadores
Release 2	Ingestão concorrente			Vizinho mais próximo (em andamento)	Desabastecimento · descarte · painel

Release 1 está entregue e em produção: as oito histórias fecham o fluxo de ponta a ponta. A Release 2 está planejada. HU-03, HU-08 e HU-05 foram quebradas em tarefas, cada uma com o teste que a protege.

E2 · TDD · TESTES POR NÍVEL

Cada história, testada em três níveis

História	Unitário (por tarefa)	Integração (com banco)	Aceitação
HU-08 Compatibilidade ABO/Rh	MatrizCompatibilidadeTest, CasosBordaCompatibilidadeTest, SelecaoFefoTest	AlocacaoAutomaticaTest	CenariosAceitacaoTest, CompatibilidadeAlocacaoTest
HU-03 Alocação FEFO	FilaPrioridadeFefoTest, SelecaoFefoTest, MotivoFalhaAlocacaoTest	FluxoRequisicaoTest	CenariosAceitacaoTest, CriteriosAceiteAlocacaoTest
HU-05 Rota de entrega	GrafoTest, DijkstraTest, DijkstraCasosSemSolucaoTest	RotaControllerTest	CenariosAceitacaoTest, DemoPontaAPontaTest

Unitário

Classe de regra isolada, sem Spring nem banco

Integração

@SpringBootTest + MockMvc, passando por JPA e H2

Aceitação

Cenários Gherkin da história em Dado / Quando / Então

E2 · TDD · A JORNADA

O teste derrubou três decisões nossas

CANCELAMENTO

Achávamos: proibir cancelar requisição já ATENDIDA.

O teste mostrou: a trava certa é o status das bolsas: RESERVADA desfaz, em trânsito não.

BOLSA VENCIDA

Achávamos: a alocação recusaria a bolsa vencida.

O teste mostrou: ela é barrada antes, na entrada do estoque; e a que vence hoje ainda vale.

ÍNDICE DE ESTOQUE

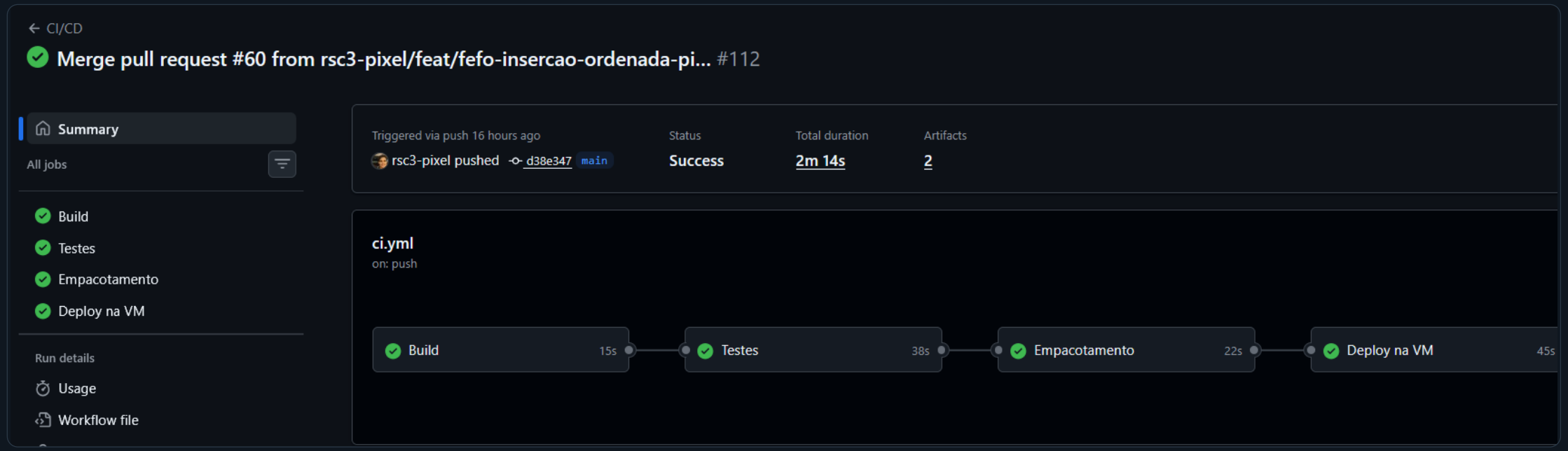
Achávamos: a colisão no hash era “rara por construção”.

O teste mostrou: Objects.hash sobre enum varia entre execuções; a chave passou a usar ordinal().

Onde não seguimos à risca: na Unidade 1 o código veio antes do teste. Na Unidade 2 os testes guiaram as regras, mas o histórico nem sempre mostra isso: os commits test: chegaram junto ou depois do feat:.

E3 · DEPLOY AUTOMATIZADO · EXECUÇÃO VERDE

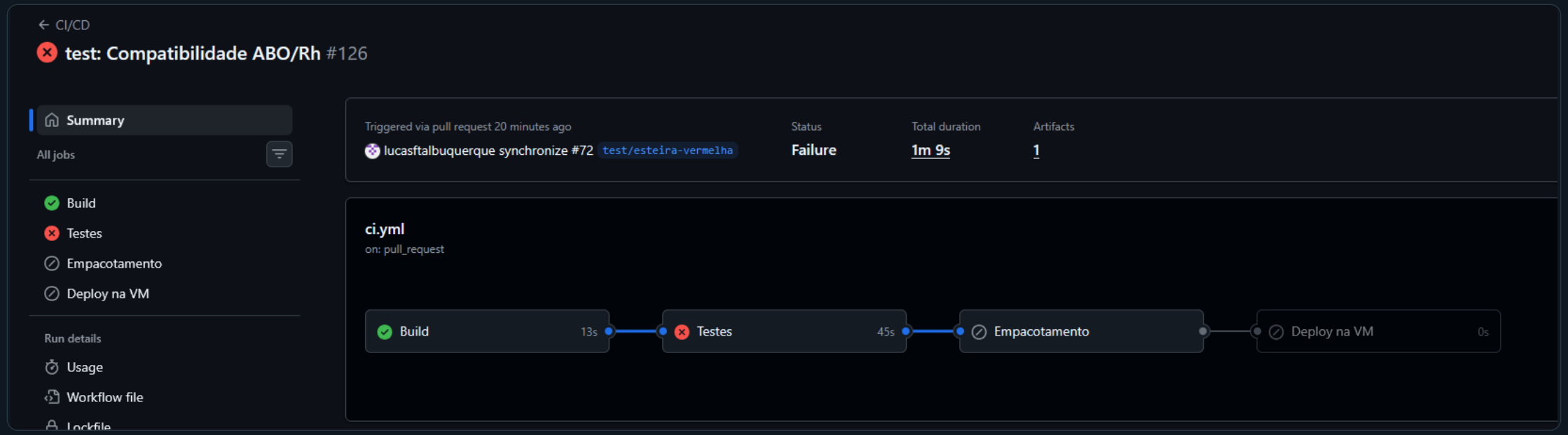
Merge na main: a esteira publica sozinha



Build → Testes → Empacotamento → Deploy na VM, em 2 min 14 s. Cada etapa só começa se a anterior passou. Desde o PR #71 a esteira também publica a **Imagem Docker no GHCR**, só na main.

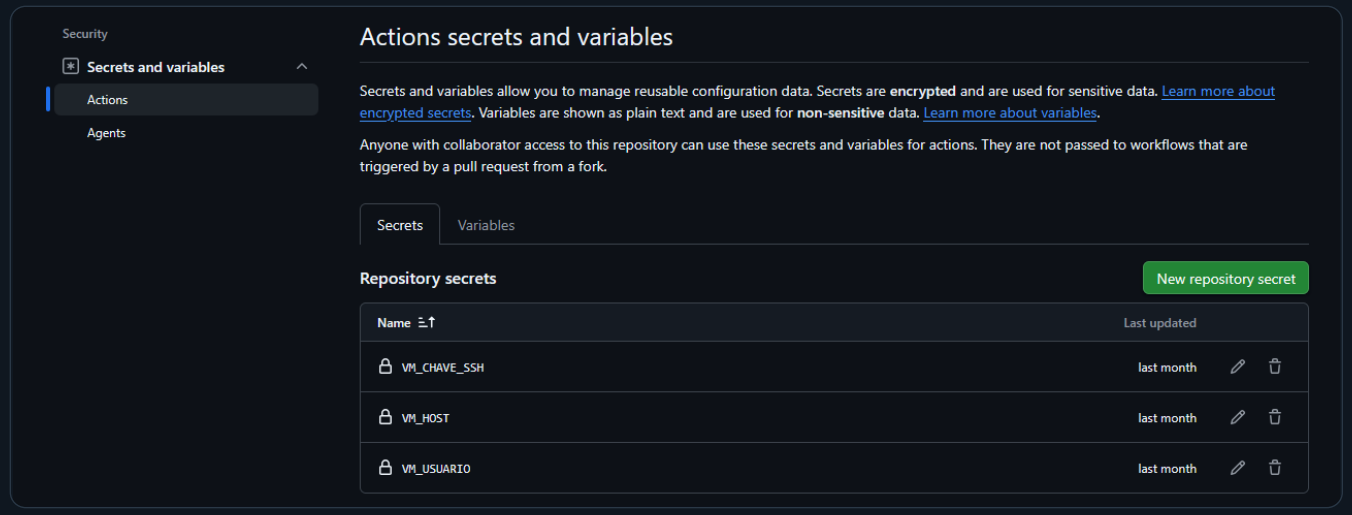
E3 · DEPLOY AUTOMATIZADO · EXECUÇÃO VERMELHA

Teste vermelho: nada é publicado



PR #72, com a regra de compatibilidade ABO/Rh quebrada de propósito: **Testes falhou**, e Empacotamento e Deploy na VM não rodaram.

Onde roda, e onde ficam os segredos



- **VM no Google Cloud**, atrás do Nginx com HTTPS
- A aplicação escuta só em **127.0.0.1:8081**: a única entrada é o Nginx
- Host, usuário e chave SSH ficam no **cofre do GitHub**, nunca no repositório
- **Rollback:** o publicar.sh guarda o jar anterior; voltar é copiar e reiniciar

E3 · O PROBLEMA QUE ENCONTRAMOS

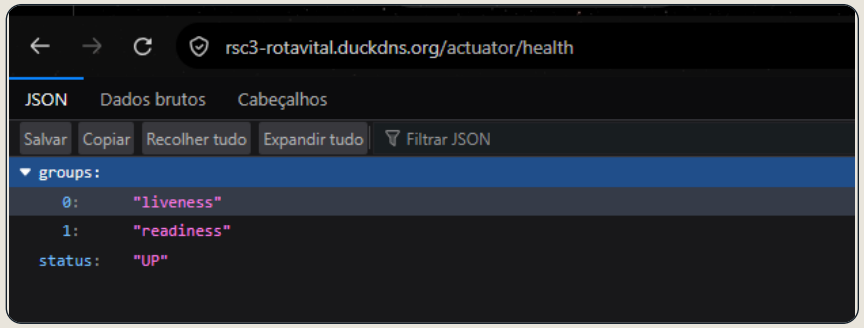
O container funciona; a VM não tem memória para ele

Memória	Medida (free -m na VM, 20/10/2026)
Total da VM	955 MB
Livre	296 MB: o resto sustenta o Nginx, o serviço da aplicação e outro projeto hospedado na mesma máquina
Container da aplicação	330 MB
Container do PostgreSQL	100 MB
Daemon do Docker	80 MB
Containers somados	**≈ 510 MB, contra 296 livres: o kernel mataria o processo mais pesado, a JVM**

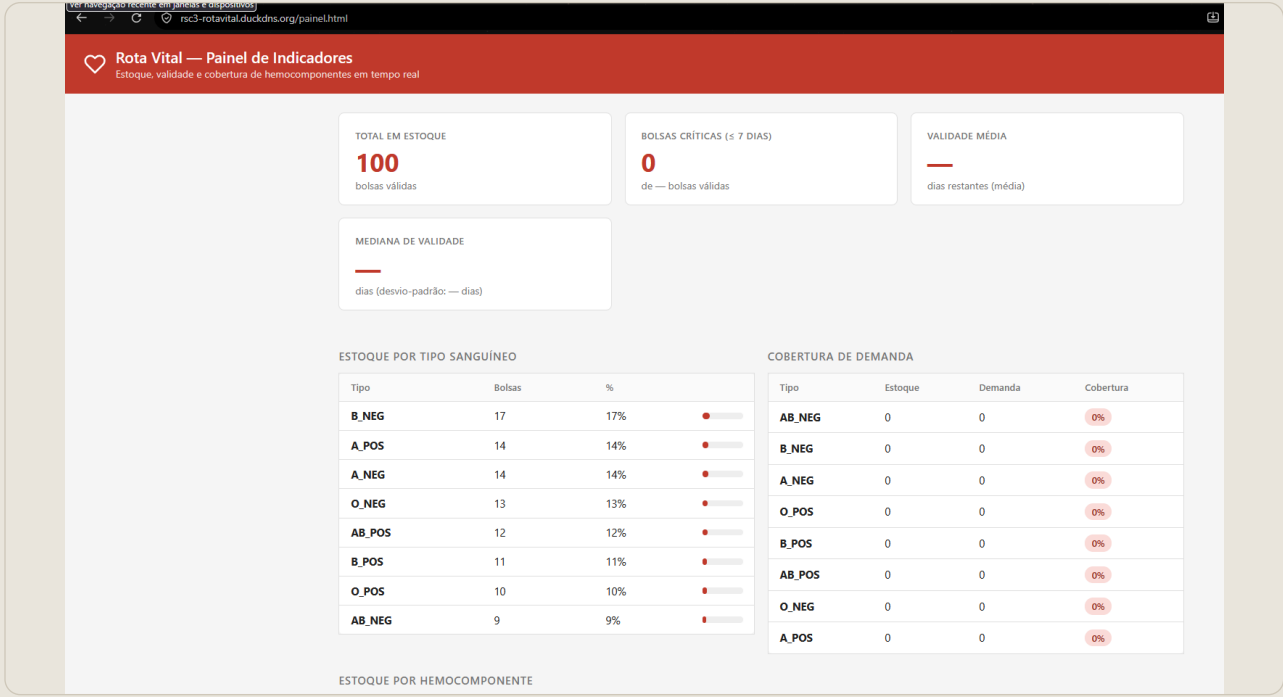
A decisão: separar construir de entregar. A esteira constrói e publica a imagem no GHCR a cada merge, versionada por commit; o deploy entrega o jar. O deploy por container entra quando houver uma VM maior, com o Nginx atual como proxy reverso.

E3 · URL PÚBLICA

No ar em rsc3-rotavital.duckdns.org



/actuator/health responde **UP**: é o que a esteira confere depois de cada deploy



/painel.html · indicadores calculados a partir do estoque (dados sintéticos)

DECISÕES

O que escolhemos, e por quê

Decisão	Por quê
JRE 21 Alpine	Executar não exige compilador; Alpine é menor e tem menos pacotes vulneráveis
Build multi-stage	A imagem final não leva Maven nem código-fonte, e nenhuma imagem existe sem os testes passarem
Volume nomeado	Os dados sobrevivem ao docker compose down, em qualquer sistema operacional
Testes por risco	Compatibilidade ABO/Rh primeiro: um erro ali oferece bolsa incompatível como válida
GitHub Flow	Grupo pequeno, uma versão no ar e deploy a cada merge, com branch curta e pull request
GitHub Actions + VM	O código já está no GitHub; a VM já tinha HTTPS e domínio, e fica sempre no ar
Segredos no cofre	Quem clona o repositório não recebe credencial nenhuma
Jar na VM, imagem no GHCR	A VM tem 296 MB livres e os containers pedem ~510 MB; a imagem fica pronta, versionada por commit

Obrigado

REPOSITÓRIO

github.com/rsc3-pixel/Projeto3-2026.2

NO AR

rsc3-rotavital.duckdns.org

Projeto acadêmico: dados sintéticos, telemetria simulada e compatibilidade ABO/Rh didática, que não substitui protocolo clínico.

Uso de IA declarado: ferramentas de IA generativa apoiaram a revisão de requisitos, a documentação e o desenvolvimento; todo o conteúdo foi revisado e validado pelo grupo.